**GROUP 7 – DOCUMENTATION**

**MILESTONE – 3 ORCHESTRATION & MONITORING**

## 1. Objective

The objective of this milestone is to automate the ETL pipeline using Apache Airflow and implement monitoring mechanisms such as logging and alerting.

The pipeline performs:
- Data extraction from CSV files
- Data cleaning and transformation
- Data storage in a database
- Automated execution using Airflow DAG
- Monitoring using logs and alerts

## 2. Dataset Description

**Orders Table**

| Column | Description |
|---|---|
| order_id | Unique order identifier |
| order_date | Date when the order was placed |
| customer_name | Name of the customer |
| state | Customer state |
| city | Customer city |

**Order Details Table**

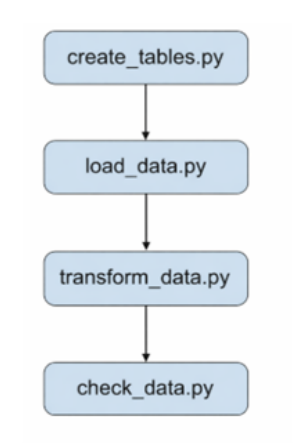
| Column | Description |
|---|---|
| order_id | Reference to Orders table |
| amount | Total order amount |
| profit | Profit generated from order |
| quantity | Quantity of items purchased |
| category | Product category |
| sub_category | Product sub category |

**Sales Target Table**

| Column | Description |
|---|---|
| month | Sales target month |
| category | Product category |
| target | Sales target value |

## 3. ETL Workflow

Airflow DAG Execution Flow:




Each stage performs a specific responsibility and executes sequentially based on defined task dependencies.

## 4. Implementation Details

### 4.1 Create Tables Script

Imports:
sqlite3 – database connection
os – directory handling
logging – execution tracking

Logic:
- Creates database folder
- Connects to SQLite
- Creates tables if not exists
- Enables foreign keys
- Logs execution steps

### 4.2 Load Data Script

Imports:
pandas – CSV handling
sqlite3 – database connection
logging – tracking

**Logic:**

**- Deletes old data (idempotent)**

```
cursor.execute("DELETE FROM order_details")
cursor.execute("DELETE FROM orders")
cursor.execute("DELETE FROM sales_target")
conn.commit()
```

**- Reads CSV files**

```
orders_df = pd.read_csv("/home/unix16/airflow/data/List of Orders.csv")
details_df = pd.read_csv("/home/unix16/airflow/data/Order Details.csv")
target_df = pd.read_csv("/home/unix16/airflow/data/Sales target.csv")
```

**- Cleans duplicates**

```
orders_df.drop_duplicates(inplace=True)
details_df.drop_duplicates(inplace=True)
target_df.drop_duplicates(inplace=True)
```

**- Loads into database**

```
orders_df.to_sql("orders", conn, if_exists="append", index=False)
details_df.to_sql("order_details", conn, if_exists="append", index=False)
target_df.to_sql("sales_target", conn, if_exists="append", index=False)
```

**4.3 Transform Data Script**

Imports:

pandas, sqlite3, logging

**Logic:**

**- Cleans orders data**

```
orders_df = pd.read_sql("SELECT * FROM orders", conn)
orders_df.dropna(subset=["order_id"], inplace=True)
```

**- Converts date format**

```
orders_df["order_date"] = pd.to_datetime(
orders_df["order_date"], dayfirst=True, errors="coerce"
)
```

**- Cleans text fields**

```
orders_df["customer_name"] = orders_df["customer_name"].str.strip()
orders_df["state"] = orders_df["state"].str.lower()
orders_df["city"] = orders_df["city"].str.lower()
```

**- Cleans numeric data**

```
details_df["amount"] = pd.to_numeric(details_df["amount"], errors="coerce")
details_df["profit"] = pd.to_numeric(details_df["profit"], errors="coerce")
details_df = details_df[details_df["quantity"] > 0]
details_df = details_df[details_df["amount"] >= 0]
details_df.dropna(inplace=True)
```

**- Creates cleaned tables**

```
orders_df.to_sql("clean_orders", conn, if_exists="replace", index=False)
details_df.to_sql("clean_order_details", conn, if_exists="replace", index=False)
target_df.to_sql("clean_sales_target", conn, if_exists="replace", index=False)
```

**Special:**

**Used exception to test email alerts**

```
raise Exception("Test Email Alert")
```

### 4.4 Check Data Script

Imports:

```
sqlite3, logging
```

**Logic:**

**- Counts records in tables**

```
cursor.execute("SELECT COUNT(*) FROM orders")
orders_count = cursor.fetchone()[0]
```

**- Displays results**

```
logging.info(f"Total Orders: {orders_count}")
```

**- Validates pipeline**

```
cursor.execute("SELECT COUNT(*) FROM order_details")
logging.info(f"Total Order Details: {cursor.fetchone()[0]}")
```

## 5. Airflow DAG

**- DAG name: infosys_etl_pipeline**

```
with DAG(
dag_id='infosys_etl_pipeline',
```

**- Uses BashOperator**

```
create_tables = BashOperator(
 task_id='create_tables',
 bash_command='python3 ~/airflow/scripts/create_tables.py'
 )
```

**- Executes scripts sequentially**

```
create_tables >> load_data >> transform_data >> check_data
```

**6. Scheduling**

**- Schedule: every 2 minutes**

```
schedule='*/2 * * * *'
```

Scheduling can be modified to:

- Daily
- Hourly
- Weekly

This removes the need for manual execution.

| Dag Run ID | Run After | State | Run Type | Trigge |
|---|---|---|---|---|
| scheduled__2026-03-19T05:58:00+00:00 | 2026-03-19 11:28:00 | Success | Scheduled | |
| scheduled__2026-03-19T05:56:00+00:00 | 2026-03-19 11:26:00 | Success | Scheduled | |
| scheduled__2026-03-19T05:54:00+00:00 | 2026-03-19 11:24:00 | Success | Scheduled | |

**7. Logging Implementation**

Logging is implemented in all scripts using the Python logging module.

Purpose of logging:

- Track each step of execution
- Maintain history of pipeline runs
- Help in debugging errors

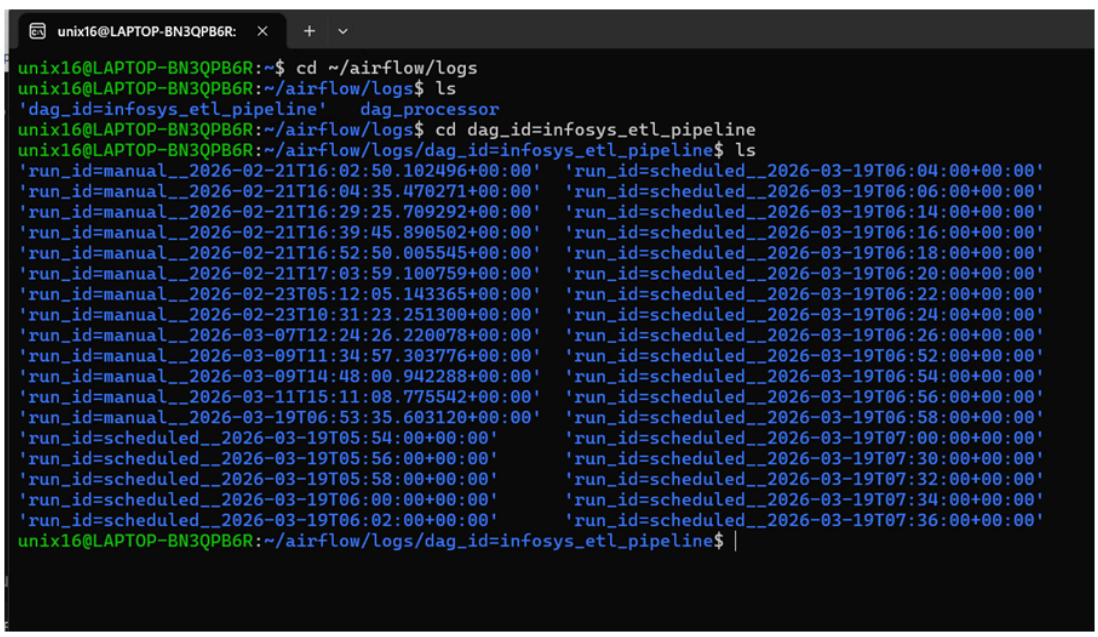
**Implementation:**

Added logging in all scripts using:
import logging
logging.basicConfig(level=logging.INFO)

**Example Logs:**
Transformation started
Clean Orders table created!
Clean Order Details table created!
Clean Sales Target table created!
Transformation completed

Logs are stored in:
~/airflow/logs/

**Outcome:**
• Complete execution history is maintained
• Easy debugging and monitoring




## 8. Alerts and Notifications

Airflow provides alerting functionality to notify failures.

**Configured email alerts in DAG:**
'email': ['mailid@gmail.com'],

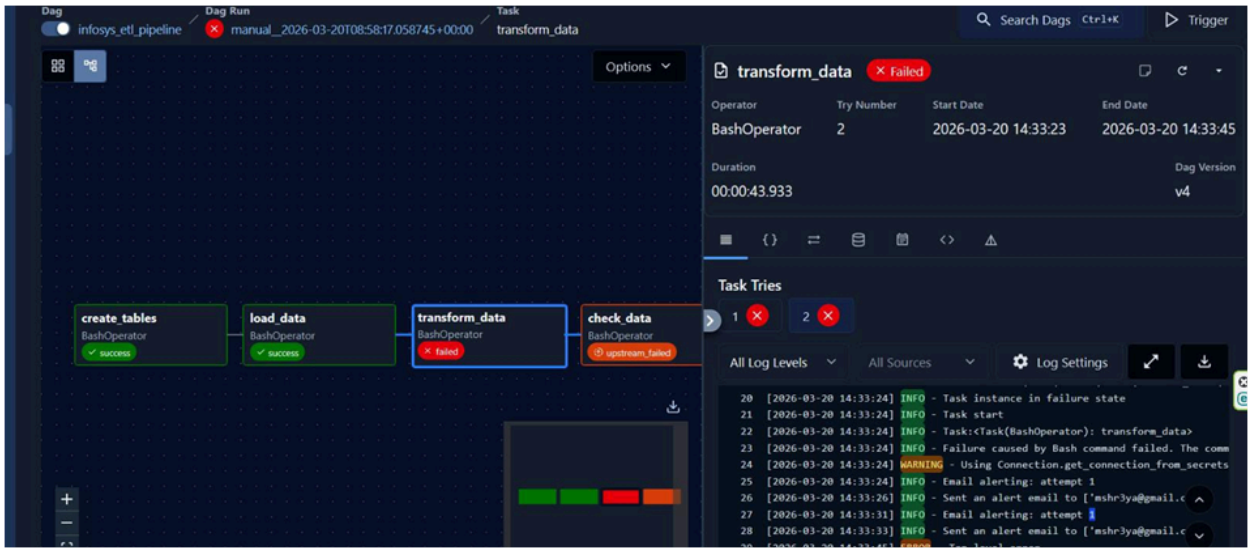
email_on_failure = True
email_on_retry = True

Alerts are triggered when:

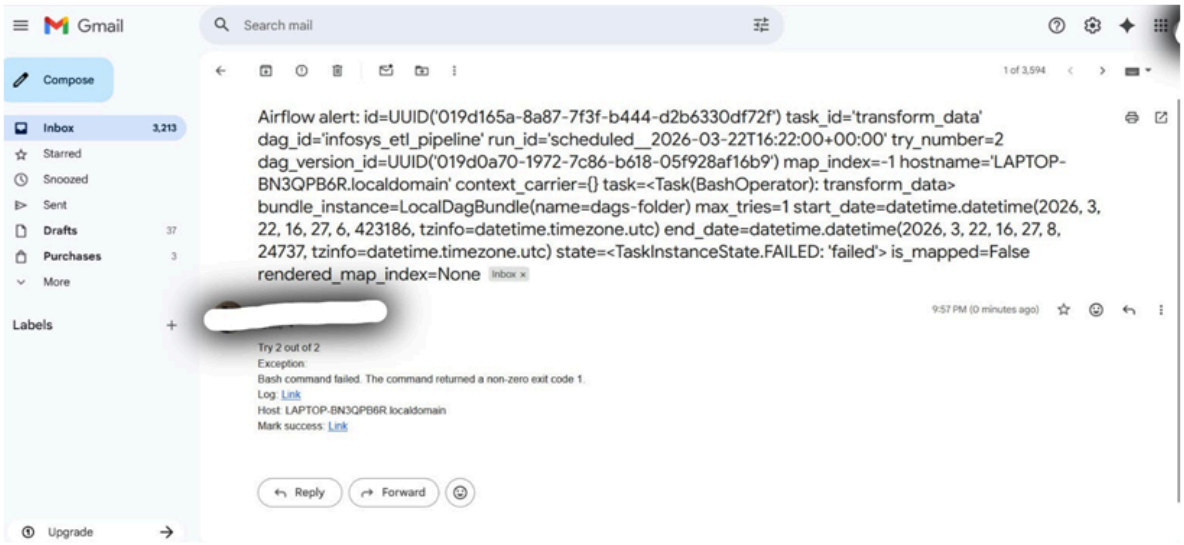
- A task fails
- A task is retried

The alert system is tested by intentionally raising an exception in the transformation step.
This ensures that failures are immediately communicated to the user.

**TASK FAILS:**




**EMAIL NOTIFICATION SENT TO MAIL:**



## 9. Monitoring using Airflow UI

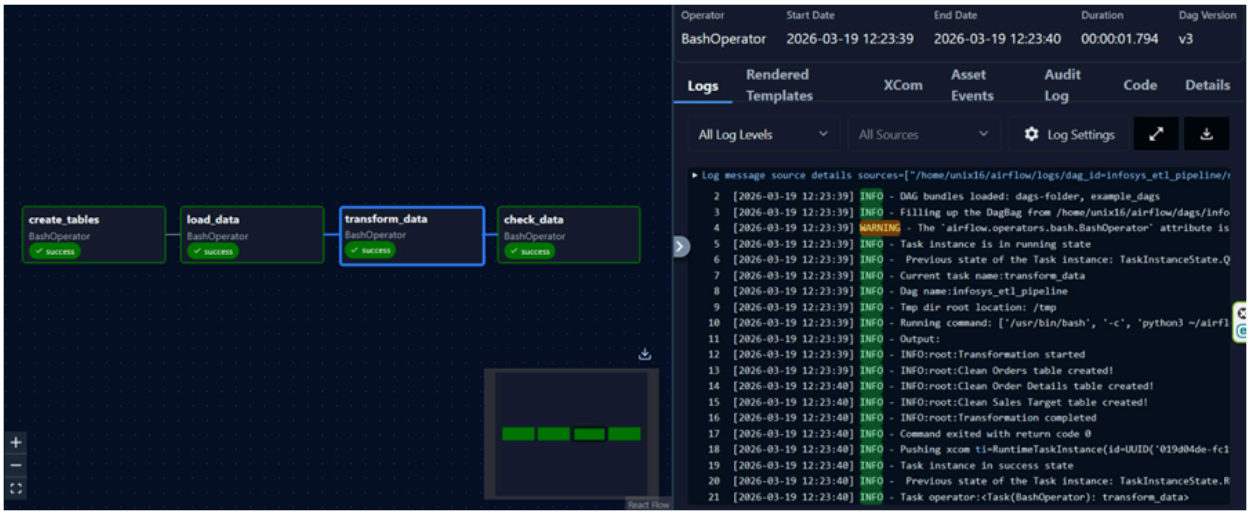
Airflow UI is used to monitor the pipeline execution.

Features:

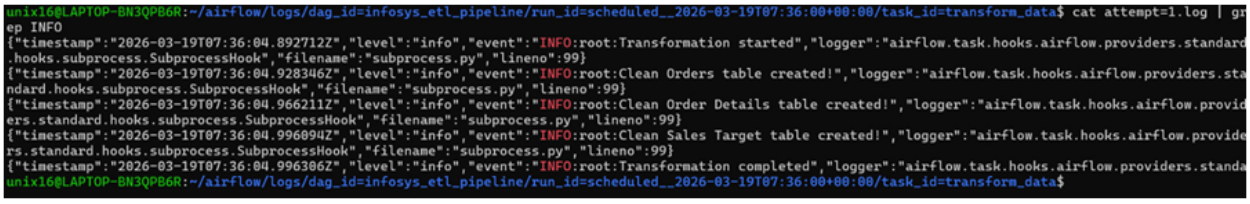
- View DAG runs
- Check task status (success/failure)
- Access logs
- Retry failed tasks

This provides real-time visibility of the ETL pipeline.

**SUCCESSFUL RUNNING IN AIRFLOW UI :**




**INFO :**




## 10. Output of Dataset

After execution, the final cleaned datasets are stored in the database.

Outputs include:

- Cleaned Orders data
- Cleaned Order Details data
- Cleaned Sales Target data

These datasets are ready for analysis and reporting.

## 11. Conclusion

The ETL pipeline is successfully automated using Airflow with monitoring and alerting.

## 12. One-Line Summary

Automated an ETL pipeline using Airflow with scheduling, logging, monitoring, and alerts.